
Modelo do Espaço Vetorial

Aula 02 — Vetores, TF-IDF e similaridade do cosseno

Projeto Integrador III ■ Ciência de Dados

FATEC — Tecnologia em Ciência de Dados

Na Aula 01

- Tokenização e vocabulário
- Matriz termo-documento
- Busca booleana (sem ordem)
- Primeira noção de TF-IDF

Hoje

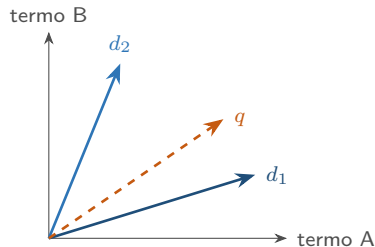
- Documentos como **veto**res
- Pesos TF-IDF completos
- **Similaridade do cosseno**
- **Ranquear** de verdade

Pergunta central

Como medir *o quanto* um documento combina com a consulta — e ordenar por isso?

O modelo do espaço vetorial

- Cada **termo** do vocabulário é uma **dimensão**.
- Cada documento (e a consulta) vira um **vetor** nesse espaço.
- **Relevância** $\approx$ proximidade entre os vetores.



$$\text{tfidf}(t, d) = \underbrace{\text{tf}_{t,d}}_{\text{freq. no documento}} \times \underbrace{\log \frac{N}{\text{df}_t}}_{\text{raridade no corpus}}$$

- **TF**: termo que aparece muito *no documento* deve pesar mais.
- **IDF**: termo que aparece em *quase todos* os documentos discrimina pouco.
- O produto premia termos **frequentes no documento** e **raros no corpus**.

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|} = \frac{\sum_t q_t d_t}{\sqrt{\sum_t q_t^2} \sqrt{\sum_t d_t^2}}$$

- Mede o **ângulo** entre os vetores — ignora o *tamanho* do documento.
- Vale **1** (mesma direção) a **0** (ortogonais, nada em comum).
- Documentos longos deixam de ser injustamente favorecidos.

Reconstruindo a matriz termo-documento

```
docs <- c(
  d1 = "recuperacao de informacao ordena documentos por relevancia",
  d2 = "o modelo de espaco vetorial representa documentos como vetores",
  d3 = "bm25 e um modelo probabilistico de ranqueamento de texto",
  d4 = "aprendizado estatistico fundamenta a recuperacao moderna",
  d5 = "o indice invertido acelera a busca em muitos documentos",
  d6 = "embeddings capturam a semantica de palavras e documentos",
  d7 = "a avaliacao mede a relevancia dos resultados da busca",
  d8 = "ciencia de dados combina estatistica e programacao"
)
tok    <- function(x) unlist(strsplit(tolower(x), "\\s+"))
tokens <- lapply(docs, tok)
vocab  <- sort(unique(unlist(tokens)))
tdm <- sapply(tokens, function(t) as.integer(table(factor(t, levels = vocab))))
rownames(tdm) <- vocab
dim(tdm)

[1] 45  8
```

Calculando os pesos TF-IDF

```
tf <- tdm
N <- ncol(tdm)
df <- rowSums(tdm > 0)
idf <- log(N / df)
w <- tf * idf # matriz TF-IDF (termos x documentos)
round(w[c("documentos", "modelo", "de"), ], 2)
```

	d1	d2	d3	d4	d5	d6	d7	d8
documentos	0.69	0.69	0.00	0	0.69	0.69	0	0.00
modelo	0.00	1.39	1.39	0	0.00	0.00	0	0.00
de	0.47	0.47	0.94	0	0.00	0.47	0	0.47

Normalizando cada documento

Dividir cada coluna pela sua norma faz todo documento virar um **vetor unitário**.

```
norm_cols <- function(m) sweep(m, 2, sqrt(colSums(m^2)), "/")
wn <- norm_cols(w)
round(colSums(wn^2), 2)           # comprimento2 de cada documento
```

d1	d2	d3	d4	d5	d6	d7	d8
1	1	1	1	1	1	1	1

A consulta também é um vetor

```
cosseno <- function(a, b) sum(a * b) / (sqrt(sum(a^2)) * sqrt(sum(b^2)))
```

```
consulta <- "modelo de recuperacao"
```

```
q <- as.integer(table(factor(tok(consulta), levels = vocab)))
```

```
qw <- q * idf
```

```
round(qw[qw > 0], 2)
```

	de	modelo recuperacao
	0.47	1.39 1.39

Ranqueando os documentos

```
scores <- apply(w, 2, function(dvec) coseno(qw, dvec))  
round(sort(scores, decreasing = TRUE), 3)
```

d1	d3	d4	d2	d6	d8	d5	d7
0.254	0.233	0.215	0.208	0.025	0.023	0.000	0.000

A ordem reflete **relevância**: d1 e d3 (sobre recuperação e modelo) lideram; d5/d7 não têm termos da consulta.

O melhor resultado

```
melhor <- names(which.max(scores))  
docs[melhor]  
  
d1  
"recuperacao de informacao ordena documentos por relevancia"
```

Forças

- Simples e eficiente
- Base de praticamente toda a RI
- Ranking interpretável

Limites

- Trata termos como **independentes**
- Não capta **sinônimos** nem semântica
- “automóvel” $\neq$ “carro”

Isso motiva, mais adiante, a **recuperação densa** (embeddings) — Fase 3.

Tarefa

- ① Reaproveitar o *corpus* de 8 documentos da Aula 01.
- ② Implementar a matriz TF-IDF e a função cosseno.
- ③ Escolher **3 consultas** e reportar o *ranking* de cada uma.

Próxima aula

- Pré-processamento de texto
- Índice invertido eficiente

Leitura

- Manning et al., *IIR* — cap. 6
- Baeza-Yates & Ribeiro-Neto — cap. 3

Obrigado!

Dúvidas? Vamos ao R.
